

JAVA

Polimorfismo, Composição & Encapsulamento

OOP PARTE 2

A segunda parte da série de OOP: como montar objetos a partir de outros objetos, como proteger o estado interno de uma classe e como o mesmo código pode se comportar de formas diferentes em runtime — tudo isso usando o catálogo de uma loja de roupas online como cenário dos exemplos.

01 Os 3 Pilares, Vistos em Conjunto

02 Composição (HAS-A)

03 Encapsulamento na Prática

04 Polimorfismo e Dynamic Dispatch

05 instanceof, Pattern Matching e var

06 Packages & Import

JavaStudiesHub

Os 3 Pilares, Vistos em Conjunto

Composição, Encapsulamento e Polimorfismo trabalhando lado a lado

Três ideias que raramente aparecem sozinhas

A primeira apostila desta série cobriu classes, objetos, construtores, herança e uma introdução ao override. Esta apostila aprofunda três pilares que, na prática, quase sempre aparecem juntos em um design de classes bem feito: **composição**, **encapsulamento** e **polimorfismo**.

Composição descreve como um objeto pode ser construído a partir de outros objetos — a relação **HAS-A** (tem um). Um pedido de compra, por exemplo, tem um produto principal, tem um cupom de desconto, tem um endereço de entrega. Em vez de empilhar tudo isso em uma única superclasse gigante, o objeto é montado a partir de peças menores e mais especializadas — o que deixa o design mais flexível e menos acoplado.

Encapsulamento é o que protege o estado interno de cada uma dessas peças: os campos ficam `private`, e a única forma de interagir com o objeto é através da interface pública que ele expõe. Quem usa a classe não precisa — e não deve — conhecer os detalhes de como ela guarda seus dados por dentro.

Polimorfismo é o que permite que o mesmo trecho de código funcione de forma diferente dependendo do tipo real do objeto em tempo de execução. Um método `calcularPrecoFinal()` chamado em uma variável do tipo `ItemDoCarrinho` pode executar uma implementação diferente conforme o objeto real por trás dela seja uma `Camiseta`, um `Calçado` ou um `Acessorio`.

```
public class Pedido { // <- Composição (HAS-A)
    private ItemDoCarrinho itemPrincipal; // Pedido TEM um item
    private CupomDeDesconto cupom; // Pedido TEM um cupom (pode ser null)

    // <- Encapsulamento: campos private, acesso via métodos
    public void aplicarCupom(CupomDeDesconto cupom) {
        this.cupom = cupom; // controla como o cupom entra no pedido
    }

    public double calcularTotal() {
        // <- Polimorfismo: calcularPrecoFinal() se comporta diferente
        // para Camiseta, Calçado e Acessorio, decidido em runtime
        double preco = itemPrincipal.calcularPrecoFinal();
        return (cupom != null) ? cupom.aplicar(preco) : preco;
    }
}
```

PILARES EM CONJUNTO

Difícilmente você usará apenas um desses três pilares isoladamente. Um bom design de classes costuma aplicar composição, encapsulamento e polimorfismo ao mesmo tempo.

Composição (HAS-A)

IS-A vs HAS-A e por que a composição costuma ser a escolha mais segura

IS-A (herança) vs HAS-A (composição)

Herança expressa uma relação **IS-A**: CalçadoEsportivo IS-A Produto — a subclasse é um tipo mais específico da superclasse, herdando toda a sua identidade. Composição expressa uma relação diferente, **HAS-A**: um Pedido HAS-A ItemDoCarrinho. Aqui o objeto contém outros objetos como partes, sem herdar identidade nenhuma deles — ele apenas os usa.

Uma boa regra prática de engenharia de software é: na dúvida, prefira composição a herança. Isso não significa banir herança — significa parar e verificar se a relação IS-A realmente faz sentido antes de usá-la. É comum, e muitas vezes desejável, combinar as duas no mesmo design: herança para expressar identidade, composição para montar funcionalidade.

```
// IS-A via herança:
public class Produto { protected String nome; protected double precoBase; }

public class Camiseta extends Produto { // IS-A Produto
// HAS-A via composição:
private Tecido tecido;
private Estampa estampa;

public Camiseta(String nome, double precoBase, Tecido tecido, Estampa estampa) {
this.nome = nome;
this.precoBase = precoBase;
this.tecido = tecido;
this.estampa = estampa;
}

// Encapsula as partes — quem usa Camiseta não precisa conhecer Tecido nem Estampa
public String descricaoCompleta() {
return nome + " (" + tecido.getTipo() + ", estampa " + estampa.getNome() + ")";
}
}
```

Por que a composição costuma ser mais segura

A herança tem um risco pouco óbvio: ela pode quebrar o encapsulamento da própria superclasse, já que a subclasse frequentemente precisa entender detalhes internos do pai para funcionar corretamente. Isso cria um acoplamento rígido — mudar a superclasse pode afetar todas as subclasses de uma vez. Outro problema comum: nem todo requisito novo se encaixa perfeitamente na hierarquia existente. Imagine que Produto tenha campos de peso e dimensões para cálculo de frete — isso faz sentido para uma Camiseta física, mas não para um VoucherDigital (um vale-presente sem envio físico). Herdar peso e dimensões nesse caso seria um erro de modelagem.

```
// Problema ao herdar algo que nem todo subtipo usa:
public class Produto { double peso, largura, altura; } // pensado para itens físicos
public class VoucherDigital extends Produto { } // nao tem peso nem dimensoes!

// Solução com composição — só quem precisa das dimensões as inclui:
public class DadosDeEnvio { double peso, largura, altura; }
public class Produto { String nome; double precoBase; } // genérico

public class ProdutoFisico extends Produto {
private DadosDeEnvio dadosDeEnvio; // HAS-A
}
public class VoucherDigital extends Produto {
private String codigoDeResgate; // sem dados de envio — correto!
}
```

Encapsulamento na Prática

Por que campos abertos são perigosos e como manter o objeto sempre válido

O risco de deixar campos abertos

Sem encapsulamento, qualquer parte do programa pode alterar diretamente o estado interno de um objeto, sem nenhuma validação. Isso traz dois problemas sérios: primeiro, o objeto pode acabar em um estado que não faz sentido — como um estoque negativo; segundo, o acoplamento fica alto demais, porque qualquer mudança futura no nome ou tipo do campo quebra todo o código que o acessava diretamente.

Encapsular é transformar o objeto em uma caixa-preta: a implementação interna pode mudar completamente, desde que a interface pública — os métodos — continue funcionando do mesmo jeito para quem usa a classe. É isso que permite melhorar e otimizar o código por dentro sem quebrar nada por fora.

```
// SEM encapsulamento – frágil:
public class Estoque {
    public int quantidade; // qualquer código pode colocar um valor negativo
}
Estoque e = new Estoque();
e.quantidade = -50; // inválido, mas nada impede!

// COM encapsulamento:
public class Estoque {
    private int quantidade;

    public Estoque(int quantidadeInicial) {
        this.quantidade = Math.max(0, quantidadeInicial); // valida na criação
    }

    public void registrarVenda(int unidades) {
        this.quantidade = Math.max(0, quantidade - unidades); // nunca fica negativo
    }

    public int getQuantidade() { return quantidade; }
}
```

Princípios práticos de encapsulamento

Princípio	Na prática
private em tudo	Todo campo começa private, sem exceção — nunca public.
Construtor validador	O construtor recebe os dados e já recusa valores inválidos, evitando objetos inconsistentes.
Setters com parcimônia	Só crie um setter quando o campo realmente precisa mudar depois da criação; prefira imutabilidade quando possível.

Princípio	Na prática
Exponha o mínimo necessário	Só torne public aquilo que quem usa a classe realmente precisa chamar.
Getters costumam ser públicos	Ler um dado é seguro; alterar sem validação, não. Por isso getters são mais liberais que setters.

REGRA DE OURO

Encapsulamento é controle total sobre o próprio estado do objeto: nenhum código externo consegue burlar as regras de negócio da classe.

Polimorfismo e Dynamic Dispatch

Como a JVM decide, em tempo de execução, qual implementação chamar

O que realmente é polimorfismo

Polimorfismo vem do grego "muitas formas": o mesmo código se comporta de maneiras diferentes dependendo do tipo real do objeto em tempo de execução. O **tipo declarado** da variável determina, em tempo de compilação, quais métodos podem ser chamados; o **tipo real** do objeto determina, em tempo de execução, qual implementação de fato roda.

Para o polimorfismo funcionar é preciso existir uma relação de herança (IS-A) entre o tipo real e o tipo declarado. A JVM resolve isso através de uma tabela virtual de métodos, mecanismo chamado de **dynamic dispatch** (ou late binding). A grande vantagem prática é a extensibilidade: um novo subtipo de produto pode ser adicionado ao catálogo sem alterar uma linha sequer do código que já trabalha com o tipo base.

```
public class Produto {
    protected String nome;
    protected double precoBase;

    public double calcularPrecoFinal() {
        return precoBase; // comportamento padrão, sem ajustes
    }
}

public class Camiseta extends Produto {
    @Override
    public double calcularPrecoFinal() {
        return precoBase; // camiseta não tem ajuste extra
    }
}

public class CalçadoImportado extends Produto {
    @Override
    public double calcularPrecoFinal() {
        return precoBase * 1.35; // acréscimo de importação
    }
}

// POLIMORFISMO: tipo declarado é Produto, tipo real varia
Produto item = obterProdutoDoCatalogo("CalçadoImportado");
item.calcularPrecoFinal(); // executa a versão de CalçadoImportado, não a de Produto
```

Factory method: criando objetos sem expor o tipo concreto

O padrão Factory Method usa polimorfismo para esconder do código cliente qual subtipo concreto está sendo criado. Um método estático recebe uma indicação (uma String, um enum) e devolve o subtipo correto — mas quem chama esse método só enxerga o tipo base. Isso significa que adicionar uma nova categoria

de produto ao catálogo exige apenas criar a subclasse e um novo caso no factory; o código que já percorre o catálogo não precisa mudar.

```
public static Produto criarProduto(String categoria, String nome, double preco) {
    return switch (categoria) {
        case "CAMISETA" -> new Camiseta(nome, preco);
        case "CALCADO_IMPORTADO" -> new CalçadoImportado(nome, preco);
        default -> new Produto(nome, preco);
    };
}

// O laço abaixo nunca muda, mesmo que novas categorias sejam criadas:
for (String categoria : categoriasDoCatalogo) {
    Produto item = criarProduto(categoria, nomeDoItem, precoDoItem); // compila como Produto
    item.calcularPrecoFinal(); // roda a versão correta em runtime
}
```


instanceof, Pattern Matching e var

Verificando o tipo real em runtime e reduzindo verbosidade na declaração de variáveis

O operador instanceof

`instanceof` testa, em tempo de execução, se um objeto pertence a um determinado tipo. Ele entra em cena quando você tem uma variável do tipo base — `Produto`, por exemplo — e precisa acessar um método específico de um subtipo, como um cálculo de imposto de importação que só existe em `CalcadoImportado`. A forma tradicional exige dois passos: testar com `instanceof` e, depois, fazer um cast manual. O Pattern Matching, disponível desde o JDK 16, une os dois passos em um só: se o teste for verdadeiro, a variável já nasce tipada como o subtipo, sem cast explícito.

```
// Forma tradicional — cast manual:
Produto item = catalogo.buscarPorId(42);
if (item instanceof CalcadoImportado) {
    CalcadoImportado calcado = (CalcadoImportado) item; // cast manual
    calcado.calcularImpostoDeImportacao();
}

// Pattern Matching (JDK 16+) — sem cast:
if (item instanceof CalcadoImportado calcado) {
    calcado.calcularImpostoDeImportacao(); // 'calcado' já é o subtipo certo
}

// Switch com pattern matching (JDK 21):
switch (item) {
    case Camiseta c -> c.exibirGuiaDeMedidas();
    case CalcadoImportado ci -> ci.calcularImpostoDeImportacao();
    default -> item.calcularPrecoFinal();
}
```

PREFIRA PATTERN MATCHING

Se `instanceof` retornou `true`, o cast é seguro por definição — então usar pattern matching elimina tanto a repetição de código quanto o risco de um `ClassCastException` por descuido.

var — inferência de tipo local

Desde o Java 10, a palavra-chave **var** instrui o compilador a inferir o tipo de uma variável local a partir da expressão usada para inicializá-la. O ganho é reduzir verbosidade, especialmente em declarações com tipos genéricos longos. O tipo continua sendo resolvido em tempo de compilação — `var` não torna o Java uma linguagem dinâmica.

`var` só pode ser usado em variáveis locais com atribuição na mesma linha: não funciona em campos de classe, em parâmetros de método, como tipo de retorno, nem atribuído diretamente a `null`, já que nesses

casos o compilador não tem como inferir o tipo.

```
// Verboso:
HashMap<String, List<Produto>> catalogoPorCategoria = new HashMap<String,
List<Produto>>();

// Com var – mesmo resultado, menos repetição:
var catalogoPorCategoria = new HashMap<String, List<Produto>>();

// var em for-each:
for (var entrada : catalogoPorCategoria.entrySet()) {
    System.out.println(entrada.getKey() + ": " + entrada.getValue().size() + " itens");
}
```

Packages & Import

Como o Java organiza classes em namespaces e evita conflitos de nome

O que é um package

Um package é um namespace que agrupa classes relacionadas — e corresponde, na prática, a uma estrutura de diretórios. A grande utilidade é evitar conflito de nomes: é perfeitamente possível ter duas classes chamadas Pedido em packages diferentes sem nenhuma ambiguidade, desde que estejam em pacotes distintos.

A convenção usa o domínio da empresa invertido como prefixo, algo como com.minhaloja.catalogo. Isso garante que o nome completo — o FQCN, ou Fully Qualified Class Name — seja globalmente único. Classes dentro do mesmo package já enxergam umas às outras com visibilidade package-private, quando nenhum modificador de acesso é declarado.

```
// package é sempre a primeira linha do arquivo:
package com.minhaloja.catalogo;

// imports vêm depois do package:
import java.util.List;
import java.util.ArrayList;

// FQCN sem import – funciona, mas é mais verboso:
java.time.LocalDate dataDoPedido = java.time.LocalDate.now();

// Regras importantes:
// 1. Apenas um package statement por arquivo
// 2. package sempre antes de qualquer import
// 3. Sem package declarado = default package (evite em projetos reais)
// 4. java.lang é importado automaticamente (String, Object, Math...)
```

Resumo da Apostila

Os conceitos essenciais desta apostila, em uma única tabela de consulta

Conceito	O que significa
Composição (HAS-A)	Objeto formado por outros objetos como partes. Costuma ser mais flexível que herança pura.
Encapsulamento	Campos private, construtores que validam dados, setters usados com parcimônia.
Caixa-preta	A implementação interna de uma classe pode mudar sem afetar quem usa sua interface pública.
Polimorfismo	Mesma chamada de método, comportamento diferente conforme o tipo real do objeto, decidido em runtime.
Dynamic dispatch	Mecanismo pelo qual a JVM escolhe, em tempo de execução, qual implementação de método executar.
Factory method	Método estático que cria e devolve o subtipo correto, escondendo o tipo concreto de quem chama.
instanceof	Testa o tipo real de um objeto em runtime. Pattern matching (JDK 16+) elimina o cast manual.
var / LVTI	Inferência de tipo em variáveis locais (JDK 10+). O tipo ainda é resolvido em tempo de compilação.
Packages	Namespace que organiza classes e evita conflito de nomes. Convenção: domínio invertido da empresa.